

当十多倍加速 仍然不该选 GPU

AccelProof 验速工坊 · 三次工程取舍

第一次转折：能运行，不等于算对

日志迁移最容易出现的错误，往往不是崩溃，而是一张看起来正常的统计表。两个成功请求的首字延迟，一个是 10，一个缺失。把缺失值填成零后，代码能跑、类型正确，均值却从 10 变成了 5。GPU 只会更快地把这个错误重复一遍。

因此，我们把统计口径先写进契约：空分组键要保留，成功状态只认 ok，首字延迟只统计成功且非空的值。模型不能改动契约，也不能放宽容差。它只能在受审计策略里选择，独立程序决定候选是否合格。故障演示是主动注入，并不伪装成模型现场出错。

检查详情 · ttft_missing_success

×

运行 v2-final-repair / attempt 0 · 失败，阻止测速与有效导出

字段	参考值	候选值	规则
mean_ttft_ms	10	5	缺失 TTFT 不能补零；[10, null] 应：10，补零变成 5

完整检查 JSON

关闭

错误检查首先展示字段、参考值、候选值和原因，再提供原始 JSON。修复不是覆盖文件后说“好了”：旧 attempt、14/18 的失败检查、新策略和复验结果都有各自的身份。

第二次转折：漂亮的比值不是运行建议

新版 500 万行记录的预热后同构 CPU/GPU 比值为 13.26X。如果只看这个指标，把默认后端切到 GPU 似乎顺理成章。但实际日常复用会启动新进程，初始化成本也由这一次任务承担。新进程对照中，CPU 花了 1.87 秒，GPU 花了 18.83 秒。



我们把结果一致性、性能测量和部署建议拆开。暖态样本回答的是“后端已经准备好时一次计算多快”；one_shot 回答“只做一次任务要等多久”；persistent_batch 则让同一进程处理两份不同输入，并计入初始化和整个批次。三个问题不能共用一个绿色成功标签。

本次 persistent_batch 的推荐为 CPU。我们没有为了让 GPU 获胜增加一个未经测量的“将来可摊薄”承诺，也没有从两组口径不同的数字推算神奇的盈亏平衡次数。每个部署模式只有一次观测，因此结论只适用于所测输入和节点。

后端建议还必须变成程序行为。v1 的复用脚本一面写 KEEP_CPU，一面无条件运行 GPU，这个矛盾在 Review 中暴露出来。v2 的日常 run 读取已审核 manifest，CPU 路径根本不调用 GPU 工具；显式 verify 才进行 CPU/GPU 双路复核。

第三次转折：验证过的文件还会变化

另一个问题更隐蔽：通过验证后，编辑候选策略，再导出。旧版会给新的错误策略附上旧的通过结论。ZIP 内部逐文件哈希即使全部正确，也只能证明“文件彼此符合这个清单”，不能证明这些文件就是当初验证过的那一组。

修复的关键不是再算一次当前哈希，而是把时间关系保存下来。验证前冻结策略、模板、判卷规则、输入和环境；从快照执行；完成后让控制器绑定结果与候选摘要；导出只能取这一份快照。验证后误改策略、契约、模板或判卷规则，都会要求重新验证。每次复用还要在新的临时目录解包，精确核对成员集合，避免上一次遗留代码影响导入。



这套机制防止意外漂移，不声称抵抗能重写整个证据目录的所有者。内容一致性与来源认证是两回事。

最后，我们也给 Agent 本身留了一条对照：固定规则。五种条件的评测把官方技能和业务技能分开，记录错误采用、token 与耗时；真正的端到端运行另行记账。当前有限策略空间里，模型未必比规则更划算。验速工坊的价值，首先是把这个事实和所有失败结果一起交付，而不是让每一轮实验都显得成功。

工具与归属：项目修订参考外部 Review，开发与材料制作使用 Codex 辅助；实验记录来自 RTX 4090。本篇属于本地评审材料，尚无公开发表或赛事接收声明。